Principal Component Analysis & Visualization

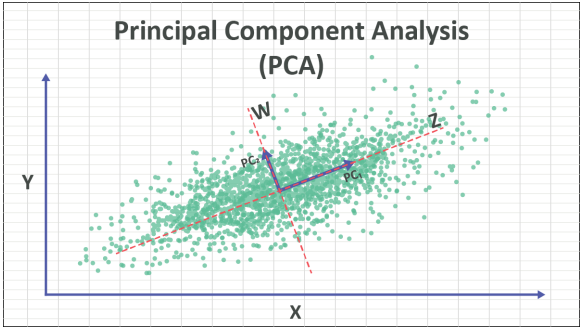
PCA

Xu Shirong

School of Economics, Xiamen University

2026

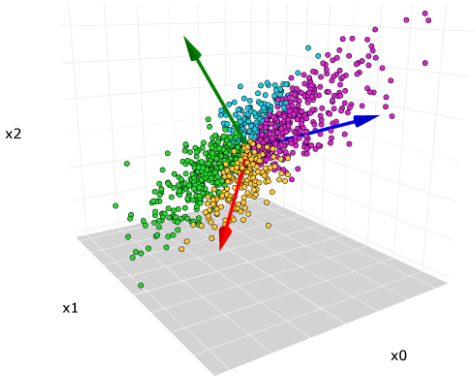
What is PCA?



PCA projects data onto directions of maximum variance.

- **Principal Component Analysis** (PCA) is a dimensionality reduction technique.
- It transforms data into a new coordinate system:
  - First axis (PC1) captures the largest variance.
  - Second axis (PC2) captures the second largest, orthogonal to PC1.
- Goal: retain as much information (variance) as possible while reducing dimensions.
- Common uses: visualization, noise reduction, feature extraction.

Geometric Interpretation of PCA



The first PC aligns with the direction of greatest spread.

- PCA finds a sequence of orthogonal directions (principal components) that maximize the variance of the projected data.
- The first PC is the line that minimizes the sum of squared perpendicular distances.
- Each subsequent PC is orthogonal to the previous ones and explains the remaining variance.
- Geometrically, it's a rotation of the axes to align with the data"s natural orientation.

Data Representation and Centering

Let the data matrix be

$$\mathbf{X} \in \mathbb{R}^{n \times d},$$

where n is the number of samples and d is the number of features.
Centering: Subtract the mean of each feature:

$$\bar{\mathbf{x}}_j = \frac{1}{n} \sum_{i=1}^{n} x_{ij}, \quad \mathbf{B} = \mathbf{X} - \mathbf{1}_n \bar{\mathbf{x}}^T.$$

Covariance matrix (sample covariance):

$$\mathbf{C} = \frac{1}{n-1} \mathbf{B}^T \mathbf{B} \in \mathbb{R}^{d \times d}.$$

Element c_{jk} measures covariance between feature j and k.

Maximizing Variance and Eigendecomposition

PCA finds directions $\mathbf{w}$ (unit vectors) that maximize the variance of projected data:

$$\mathbf{w}_1 = \underset{\|\mathbf{w}\|=1}{\operatorname{argmax}}\ \mathbf{w}^T \mathbf{C} \mathbf{w}.$$

Subsequent components are orthogonal to previous ones.
Solve the eigenvalue problem for $\mathbf{C}$:

$$\mathbf{C}\mathbf{v}_i = \lambda_i \mathbf{v}_i, \quad i = 1, \ldots, d,$$

with eigenvalues $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_d \geq 0$ and corresponding eigenvectors $\mathbf{v}_i$. The eigenvectors $\mathbf{v}_i$ are the **principal component directions** (loadings).

Dimensionality Reduction and SVD

Choosing the number of components k:

$$\mathrm{CPV}(k) = \frac{\sum_{i=1}^{k} \lambda_i}{\sum_{i=1}^{d} \lambda_i} \quad (\text{e.g.}, \ > 0.85).$$

Projection matrix: $\mathbf{W}_k = [\mathbf{v}_1 \ \mathbf{v}_2 \ \cdots \ \mathbf{v}_k] \in \mathbb{R}^{d \times k}$.
Projection (scores):

$$\mathbf{T} = \mathbf{B}\mathbf{W}_k \in \mathbb{R}^{n \times k}.$$

After PCA, we transform the data of shape $n \times p$ into $n \times k$, where k is significantly smaller than p.

Reconstruction from Principal Components

Given the scores $\mathbf{T} \in \mathbb{R}^{n \times k}$ and the loading matrix $\mathbf{W}_k \in \mathbb{R}^{d \times k}$, we can **reconstruct** (an approximation of) the centered data:

$$\hat{\mathbf{B}} = \mathbf{T}\mathbf{W}_k^T \quad \in \mathbb{R}^{n \times d}.$$

Adding back the feature means gives the reconstructed original data:

$$\hat{\mathbf{X}} = \hat{\mathbf{B}} + \mathbf{1}_n \bar{\mathbf{x}}^T.$$

- If $k = d$ (all components retained), $\mathbf{W}_d$ is orthogonal and the reconstruction is exact: $\hat{\mathbf{X}} = \mathbf{X}$.
- If $k < d$, the reconstruction is the best rank-k approximation of $\mathbf{X}$ in the least-squares sense (by the Eckart–Young theorem).
- The reconstruction error $\|\mathbf{X} - \hat{\mathbf{X}}\|_F^2$ equals the sum of the discarded eigenvalues $\sum_{i=k+1}^{d} \lambda_i$.

This is the mathematical foundation behind applications like image compression and denoising: we keep only the first k principal components, discard the rest, and recover a close

PCA Toy Example

Original data (height in cm, weight in kg):

Sample	Height (x)	Weight (y)
1	170	66
2	180	78
3	175	72
4	185	84
5	165	60
6	175	72
7	190	90
8	160	54
9	180	78
10	170	66

$$\bar{x} = 175, \quad \bar{y} = 72$$

Centered data B: subtract means

$$\mathbf{B} = \begin{bmatrix} -5 & -6 \\ 5 & 6 \\ 0 & 0 \\ 10 & 12 \\ -10 & -12 \\ 0 & 0 \\ 15 & 18 \\ -15 & -18 \\ 5 & 6 \\ -5 & -6 \end{bmatrix}$$

PCA Toy Example

Covariance matrix $\mathbf{C} = \frac{1}{n-1}\mathbf{B}^T\mathbf{B}$ with $n = 10$:

$$\mathbf{B}^T\mathbf{B} = \begin{bmatrix} \sum x_i^2 & \sum x_i y_i \\ \sum x_i y_i & \sum y_i^2 \end{bmatrix} = \begin{bmatrix} 750 & 900 \\ 900 & 1080 \end{bmatrix}$$

$$\mathbf{C} = \frac{1}{9}\begin{bmatrix} 750 & 900 \\ 900 & 1080 \end{bmatrix} = \begin{bmatrix} \frac{250}{3} & 100 \\ 100 & 120 \end{bmatrix} \approx \begin{bmatrix} 83.33 & 100 \\ 100 & 120 \end{bmatrix}$$

Principal Component Extraction and Projection

Eigenvalues of $\mathbf{C}$:

$$\det(\mathbf{C} - \lambda \mathbf{I}) = \left(\frac{250}{3} - \lambda\right)(120 - \lambda) - 100^2 = 0$$

$$\Rightarrow \lambda^2 - \left(120 + \frac{250}{3}\right)\lambda + \left(\frac{250}{3} \cdot 120 - 10000\right) = 0$$

Since the data are perfectly collinear, one eigenvalue is zero. Solving gives

$$\lambda_1 = \frac{1830}{9} = 203.\overline{3}, \qquad \lambda_2 = 0.$$

Thus the first principal component captures 100% of the variance.
First eigenvector $\mathbf{v}_1$ (unit vector): From $(\mathbf{C} - \lambda_1 \mathbf{I})\mathbf{v}_1 = 0$. Normalized:

$$\mathbf{v}_1 = \left(\frac{5}{\sqrt{61}}, \frac{6}{\sqrt{61}}\right) \approx (0.640, 0.768).$$

Principal Component Extraction and Projection

Projection onto first PC (scores) $\mathbf{t} = \mathbf{B}\mathbf{v}_1$: $t_i = \frac{5x_i+6y_i}{\sqrt{61}}$ where (x_i, y_i) are centered values.

Sample	t_i
1	$-\sqrt{61} \approx -7.81$
2	$\sqrt{61} \approx 7.81$
3	0
4	$2\sqrt{61} \approx 15.62$
5	$-2\sqrt{61} \approx -15.62$
6	0
7	$3\sqrt{61} \approx 23.43$
8	$-3\sqrt{61} \approx -23.43$
9	$\sqrt{61} \approx 7.81$
10	$-\sqrt{61} \approx -7.81$

- Look at those positive t_i, samples includes 2,4,7,9 samples.
- Using 1-dimensional data to represent 2-dimensional information.
- Address colinearity problem.
- **Question**: Can we reconstruct the original data (Height, Weight)?

Mathematical Foundation: Centering and Covariance

```
import numpy as np
X = np.array([
    [170, 66],
    [180, 78],
    [175, 72],
    [185, 84],
    [165, 60],
    [175, 72],
    [190, 90],
    [160, 54],
    [180, 78],
    [170, 66]
])
# Center the data
X_centered = X - np.mean(X, axis=0)
# Compute covariance matrix
cov_matrix = np.cov(X_centered, rowvar=
    False)
```

- PCA starts by centering the data: subtract the mean of each variable.
- The covariance matrix Σ captures relationships between features:

$$\Sigma_{ij} = \mathrm{Cov}(x_i, x_j)$$

- For p features, Σ is a $p \times p$ symmetric matrix.
- Diagonal entries are variances; off-diagonals are covariances.

Eigenvalue Decomposition for PCA

```
# Eigendecomposition of covariance
eigenvals, eigenvecs = np.linalg.eig(
    cov_matrix)

# Sort by eigenvalue (descending)
idx = np.argsort(eigenvals)[::-1]
eigenvals = eigenvals[idx]
eigenvecs = eigenvecs[:, idx]

# Principal components
PC1 = eigenvecs[:, 0]
PC2 = eigenvecs[:, 1]
```

- Eigenvectors of Σ give the **principal component directions**.
- Corresponding eigenvalues indicate the **variance explained** by each PC.
- Larger eigenvalue $\Rightarrow$ more important PC.
- PCs are orthogonal: $\mathbf{v}_i \cdot \mathbf{v}_j = 0$ for $i \neq j$.

Choosing the Number of Principal Components

```
# Explained variance ratio
explained_ratio = eigenvals / np.sum(
    eigenvals)
# Cumulative variance
cumulative = np.cumsum(explained_ratio)
# Plot explained variance and cumulative
    variance
print(explained_ratio,cumulative)
```

```
In [24]: explained_ratio = eigenvals / np.sum(eigenvals)
    ...: # Cumulative variance
    ...: cumulative = np.cumsum(explained_ratio)
    ...: # Plot
    ...: print(explained_ratio,cumulative)
    ...:
[1. 0.] [1. 1.]
```

Scree plot showing cumulative variance.

- Use **explained variance ratio** to decide how many PCs to retain.
- Common criteria: Keep enough PCs to explain 80% variance.

Step-by-Step PCA Algorithm

```python
from sklearn.decomposition import PCA

# 1. Standardize data (if features have
    different scales)
# 2. Create PCA object
pca = PCA(n_components=2)

# 3. Fit and transform
X_pca = pca.fit_transform(X)

# Access components and explained
    variance
print(pca.components_)
print(pca.explained_variance_ratio_)
```

Summary:

1. Center (and optionally scale) the data.
2. Compute covariance matrix or perform SVD.
3. Obtain eigenvalues and eigenvectors.
4. Sort eigenvectors by descending eigenvalues.
5. Choose top k eigenvectors.
6. Project data: $\mathbf{Z} = \mathbf{X}_{\text{cent}}\mathbf{W}$.

Scikit-learn provides a convenient implementation.

Advantages and Disadvantages of PCA

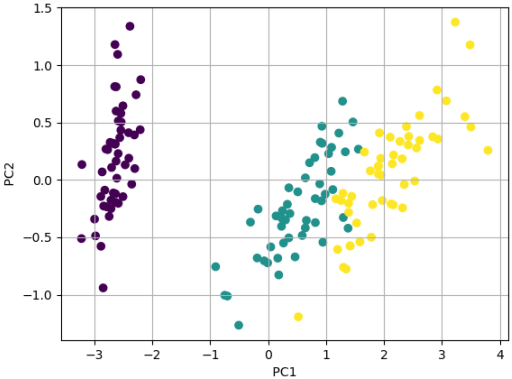
Advantages:

- Reduces dimensionality, mitigates curse of dimensionality.
- Removes multicollinearity (features become uncorrelated).
- Noise reduction: later PCs often capture noise.
- Fast and interpretable (linear combinations).
- Works well for visualization (2D/3D).

Disadvantages:

- **Linear** technique – cannot capture nonlinear structures.
- Principal components are **hard to interpret** (combinations of all original features).
- Sensitive to **scaling**: variables must be standardized.
- Assumes **normality** and linear relationships for optimal performance.
- Outliers can heavily influence results.

Application: PCA for Data Visualization (Iris Dataset)



Iris data projected onto first two PCs.

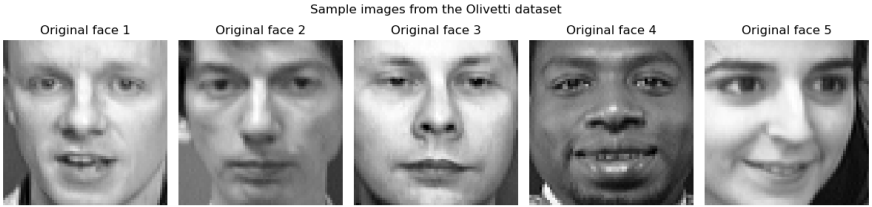
```
from sklearn.datasets import load_iris
iris = load_iris()
X = iris.data
y = iris.target

pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)

plt.scatter(X_pca[:,0], X_pca[:,1],
            c=y, cmap="viridis")
plt.xlabel("PC1")
plt.ylabel("PC2")
```

Application: PCA for Data Visualization (Human Face Dataset)

```
from sklearn.datasets import fetch_olivetti_faces
print("Loading Olivetti faces dataset...")
faces = fetch_olivetti_faces(shuffle=True, random_state=42)
X = faces.data          # shape (400, 4096), each row is a 64x64 face flattened
n_samples, n_features = X.shape
fig, axes = plt.subplots(1, 5, figsize=(12, 3))
for i in range(5):
    axes[i].imshow(X[i].reshape(64, 64), cmap="gray")
    axes[i].set_title(f"Original face {i+1}")
    axes[i].axis("off")
```



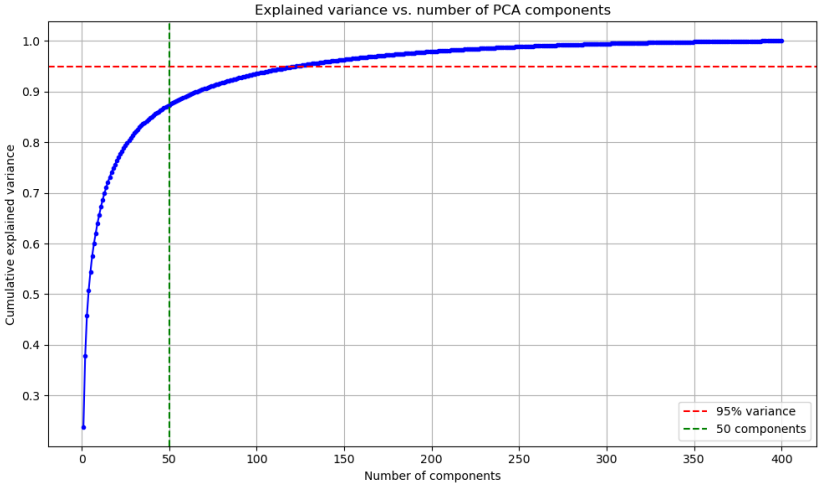
Application: PCA for Data Visualization (Iris Dataset)

```
full_pca = PCA(n_components=None)
full_pca.fit(X)
cumulative_variance = np.cumsum(full_pca.
    explained_variance_ratio_)

plt.figure(figsize=(10, 6))
plt.plot(range(1, len(cumulative_variance)+1),
     cumulative_variance, "b-", marker="o",
    markersize=3)
plt.axhline(y=0.95, color="r", linestyle="--",
     label="95% variance")
plt.axvline(x=50, color="g", linestyle="--",
    label="50 components")
plt.xlabel("Number of components")
plt.ylabel("Cumulative explained variance")
plt.title("Explained variance vs. number of
    PCA components")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

- In the original dataset, each image is of size 64×64 (high-dimensional)
- Using PCA, we can store the data with most information and less memory.

Choosing Appropriate Number of Components



For illustration, we select 50 as the case.

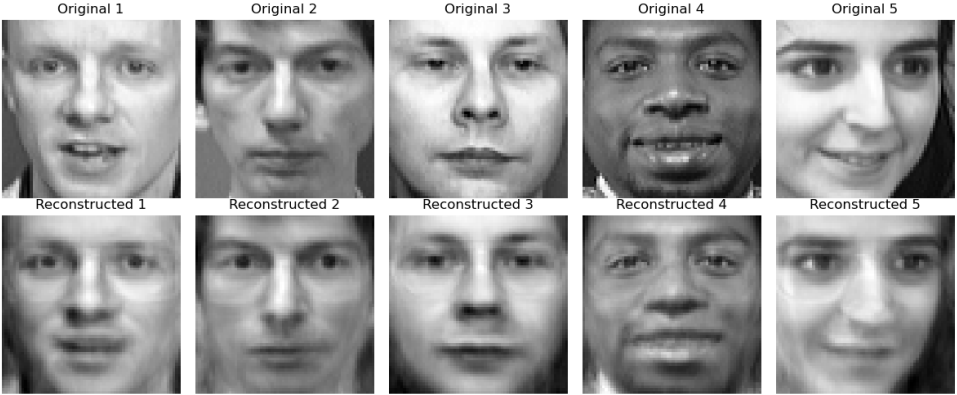
PC number = 50

```
n_components = 50
pca = PCA(n_components=n_components, whiten=False)
pca.fit(X)
print(f"PCA training completed with {n_components} components")
print(f"Cumulative explained variance: {pca.explained_variance_ratio_.sum():.3f}")
```

- PCA training completed with 50 components
- Cumulative explained variance: 0.873
- 50 components explain 87.3% variance of original 4096 dimensional data

Reconstruction of Images

```
X_test = X[:5] # Reduce dimensionality (project onto principal components)
X_test_pca = pca_loaded.transform(X_test) # Reconstruct the images from the reduced
    representation
X_test_reconstructed = pca_loaded.inverse_transform(X_test_pca)
```

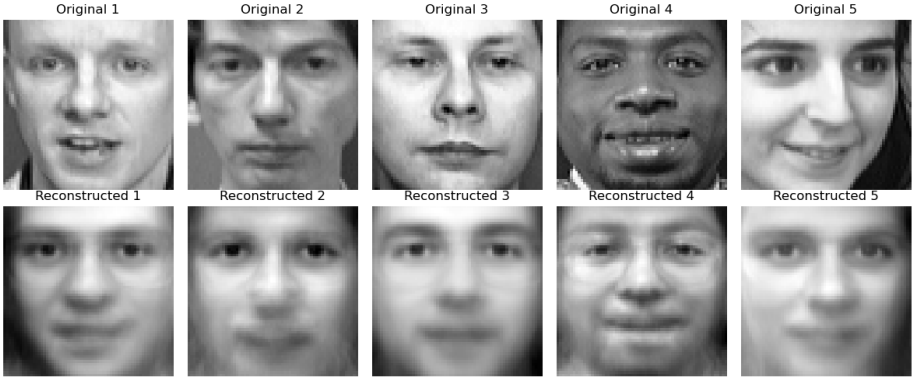


Lower Dimension of Data

```
n_components = 10
pca = PCA(n_components=n_components, whiten=False)
pca.fit(X)
X_test = X[:5]# Reduce dimensionality (project onto principal components)
X_test_pca = pca.transform(X_test)
# Reconstruct the images from the reduced representation
X_test_reconstructed = pca.inverse_transform(X_test_pca)
fig, axes = plt.subplots(2, 5, figsize=(12, 5))
for i in range(5):
    # Original image (top row)
    axes[0, i].imshow(X_test[i].reshape(64, 64), cmap="gray")
    axes[0, i].set_title(f"Original {i+1}")
    axes[0, i].axis("off")

    # Reconstructed image (bottom row)
    axes[1, i].imshow(X_test_reconstructed[i].reshape(64, 64), cmap="gray")
    axes[1, i].set_title(f"Reconstructed {i+1}")
    axes[1, i].axis("off")
plt.tight_layout()
```

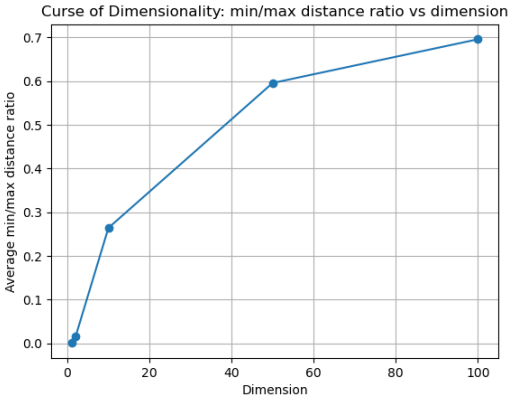
Lower Dimension of Data-Result



Much information is lost. A black person is reconstructed as a white one.

```python
import numpy as np
import matplotlib.pyplot as plt
from scipy.spatial.distance import pdist, squareform
def distance_ratio(n_points=1000, dims=[1, 2, 10, 50, 100]):
    """
    Generate random points in different dimensions and compute the
    average ratio of nearest neighbor distance to farthest neighbor distance.
    """
    ratios = []
    for d in dims:
        # Generate n_points uniformly in [0,1]^d
        X = np.random.rand(n_points, d)
        # Compute pairwise Euclidean distances
        dist_matrix = squareform(pdist(X, metric="euclidean"))
        # Ignore diagonal (distance to self)
        np.fill_diagonal(dist_matrix, np.nan)
        # Nearest neighbor distance
        min_dist = np.nanmin(dist_matrix, axis=1)
        # Farthest neighbor distance
        max_dist = np.nanmax(dist_matrix, axis=1)
        # Average ratio
        ratio = np.mean(min_dist / max_dist)
        ratios.append(ratio)
        print(f"Dimension {d}: avg min/max distance ratio = {ratio:.4f}")
    return ratios
```

Distance Metric Fails




```
dims = [1, 2, 10, 50, 100]
ratios = distance_ratio(dims=dims)
# Plot the results
plt.plot(dims, ratios, marker="o")
plt.xlabel("Dimension")
plt.ylabel("Average min/max distance
    ratio")
plt.title("Curse of Dimensionality:
    min/max distance ratio vs
    dimension")
plt.grid(True)
plt.show()
```

- As dimension increases, given a fixed sample size $n = 1000$, the ratio of the minimal distance to the maximal distance increases.

Example: Cat vs Dog Text Data

```
import numpy as np
from sklearn.feature_extraction.text import
    TfidfVectorizer
documents = ["My cat likes to sleep on the
    soft sofa in the living room, and the cat
    sleeps there every afternoon.",
"The small cat enjoys sleeping on the warm
    sofa, and the cat often sleeps quietly in
    the living room.",
"My lazy cat always sleeps on the sofa in the
    living room, because the cat likes warm
    and quiet places.",
"My dog likes to play fetch in the green park,
     and the dog runs after the ball every
    evening.",
"The happy dog enjoys playing fetch in the
    park, and the dog runs quickly to catch
    the ball.",
"My energetic dog always plays fetch in the
    park, and the dog runs fast after the ball
    ."]
```

- We create a small text dataset.
- First three sentences describe **cats**.
- Last three sentences describe **dogs**.
- TF-IDF converts each sentence into a high-dimensional vector.
- Each word becomes a feature.

Distances in the high-dimensional word space

```
from scipy.spatial.distance import pdist
from scipy.spatial.distance import squareform

vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(documents).
    toarray()
# Original distance matrix
dist_original = squareform(
    pdist(X, metric="euclidean")
)

np.set_printoptions(precision=2, suppress=True
    )

print("Original distance matrix:\n")
print(dist_original)
```

- We compute pairwise distances between all sentences.
- Distances are calculated in the **original TF-IDF space**.
- This space can be very high dimensional.
- Distance relationships may be noisy because the data is sparse.

Distance Result

```
[[0.   0.95 0.91 1.15 1.22 1.25]
 [0.95 0.   0.9  1.25 1.16 1.24]
 [0.91 0.9  0.   1.23 1.25 1.2 ]
 [1.15 1.25 1.23 0.   0.91 0.86]
 [1.22 1.16 1.25 0.91 0.   0.93]
 [1.25 1.24 1.2  0.86 0.93 0.  ]]
```

- Within-Group distance are smaller than between group distance, but very close.
- What will happen if we use principal components for distance measure.

Distances After PCA

```python
from sklearn.decomposition import PCA

# PCA
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)

# PCA distance matrix
dist_pca = squareform(
    pdist(X_pca, metric="euclidean")
)

print("\nPCA distance matrix:\n")
print(dist_pca)
```

- PCA projects the data into a **low-dimensional space**.
- It keeps the directions with largest variance.
- Sentences about cats and dogs become easier to separate.
- Distances after PCA often better reflect the underlying structure.

Distance Result

```
[[0.   0.76 0.29 0.93 1.16 1.01]
 [0.76 0.   0.48 1.2  0.93 1.09]
 [0.29 0.48 0.   1.02 1.06 1.02]
 [0.93 1.2  1.02 0.   0.68 0.26]
 [1.16 0.93 1.06 0.68 0.   0.43]
 [1.01 1.09 1.02 0.26 0.43 0.  ]]
```

- Groups are more separated.